

罰單超速處理系統 開發計畫

1. 違規處理子系統

功能描述

- 負責接收來自智慧燈桿的違規數據，完成分類處理，並分流至其他子系統。

開發計畫

1. 數據接收與格式化：

- 實現 API 接口接收智慧燈桿數據，包括車牌號碼、速度、地點等。
- 格式化接收到的數據，並存入 `EventBasicInfo` 表。
- 使用工具：Express.js（後端）、RESTful API。

2. 數據分流：

- 將數據按車牌號碼傳遞至車籍驗證子系統。
- 傳遞影像數據至 AI 辨識子系統。
- 若數據不完整，則記錄至異常日誌。

技術實現

- 前端：React.js 與表單驗證（檢查數據是否完整）。
 - 後端：Node.js 使用 Fastify 或 Express.js 實現。
-

2. 車籍驗證子系統

功能描述

- 驗證車牌號碼的合法性，並查詢車主相關信息。

開發計畫

1. 車牌驗證：

- 整合台灣車牌資料庫 API，檢查車牌號碼是否有效。
- 使用假數據或模擬 API 進行測試。

2. 數據儲存：

- 驗證結果存入 `VehicleInfo`，包括車主姓名、車型等。

技術實現

- 後端：
 - 實現數據查詢邏輯，若數據無效則返回錯誤狀態。
 - 使用 SQL 查詢語句優化車牌檢索。
 - 測試工具：
 - Postman 測試數據請求。
-

3. AI 辨識子系統

功能描述

- 利用 **Gemini API** 對違規影像進行車牌號碼、車型與顏色的辨識。

開發計畫

1. 影像數據傳輸：
 - 設計 API 將影像數據傳送至 Gemini API。
 - 接收辨識結果並格式化。
2. 錯誤處理：
 - 對於辨識失敗的數據（如影像模糊），記錄錯誤原因並標記狀態為「待審核」。
3. 數據儲存：
 - 成功辨識的結果存入 **AIRecognition** 表。

技術實現

- 使用 Axios 或 Fetch API 與 Gemini API 進行數據交互。
 - 設計異步函數處理 API 響應，確保高效辨識。
-

4. 人工辨識子系統

功能描述

- 針對 AI 無法辨識的案件進行人工審核，並記錄處理歷程。

開發計畫

1. 案件接收與顯示：

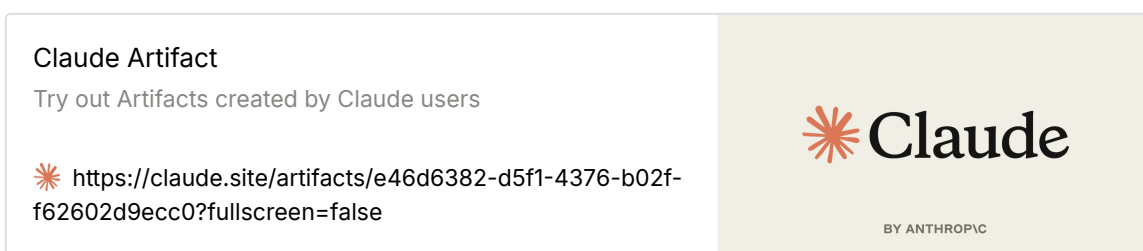
- 從 `ProcessingLog` 提取待審核案件，顯示於介面上。
- 支持影像查看與數據填寫。

2. 審核與數據提交：

- 允許人工輸入車牌號碼、車型和顏色。
- 更新審核結果至 `ProcessingLog` 和 `EventBasicInfo`。

技術實現

- 前端：
 - 實作畫面預覽



- 使用 React.js 開發可互動的案件審核界面【111+source】。
- 支持影像預覽（使用 `<img>`）。
- 檔案：

`traffic-officer-interface.tsx`

- 後端：
 - 設計更新接口，將人工輸入的數據寫入資料庫。

5. 罰單生成子系統

功能描述

- 根據確認的違規案件數據生成罰單，並通知車主。

開發計畫

1. 罰單生成：

- 提取經確認的違規案件數據，生成罰單編號與罰鍰金額。

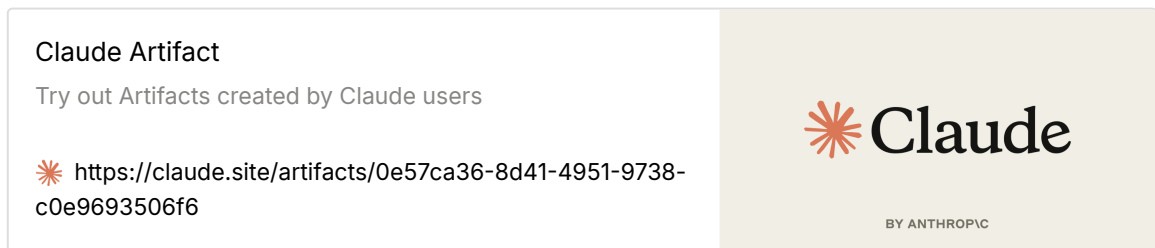
- 將罰單數據存入 `TicketInfo` 。

2. 車主通知：

- 實現通知功能（SMS 或 Email API）。
- 更新通知狀態至 `TicketInfo` 。

技術實現

- 後端：
 - 實作畫面預覽



- 使用 Nodemailer 實現 Email 通知。
- 設計 API 驗證通知是否成功。
- 檔案

[violation-public-interface.tsx](#)

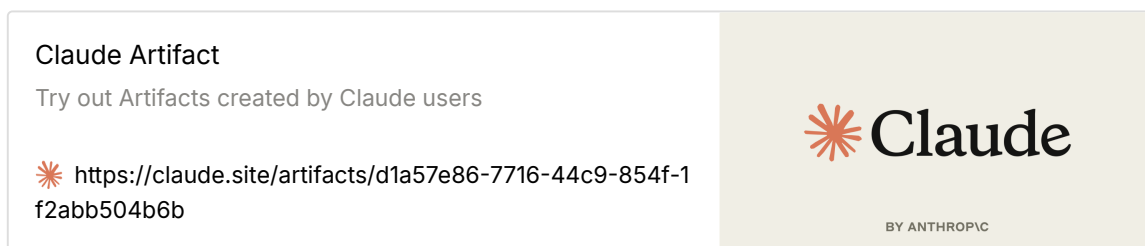
- 前端：
 - 增加罰單預覽與列印功能【112+source】。

6. 全國違規事件管理儀表板子系統：台北城市儀表板

前端

- 描述：
 - 以 React.js 建立的前端，使用 Recharts 或 Chart.js 生成數據分析圖表【110+source】。
 - 直接代入來自後端的數據，實現即時數據分析與展示。
- 實現方式：
 1. 使用 `ResponsiveContainer` 確保圖表能適應不同解析度的螢幕。

2. 整合來自後端的 API 數據，通過 Axios 或 Fetch API 獲取。
 3. 將數據映射至 Recharts 的 `LineChart`、`BarChart` 等組件進行渲染。
- 前端功能：
 - 實作畫面預覽



- 違規趨勢分析：展示超速、闖紅燈等違規數據的時間趨勢。
- 區域分布分析：以分布圖或地圖方式呈現不同地區的違規情況。
- 違規類型分析：用柱狀圖和圓餅圖展示不同違規類型的分布。
- 參考檔案(包括可以有哪些欄位、資料分析)

[traffic-dashboard.tsx](#)

後端

- 描述：
 - 撰寫 SQL 聚合查詢，從 `EventBasicInfo`、`TicketInfo` 和 `ProcessingLog` 提取數據，進行統計與篩選，並返回前端。
- 實現方式：
 1. 使用 PostgreSQL 整合數據源。
 2. 撰寫高效 SQL 查詢語句，如下：

```
SELECT
    COUNT(*) AS total_events,
    COUNT(CASE WHEN violation_type = '超速' THEN 1 END)
AS speeding_events,
    COUNT(CASE WHEN violation_type = '闖紅燈' THEN 1 END)
AS red_light_events,
```

```
area,  
    DATE_PART('month', violation_time) AS month  
FROM EventBasicInfo  
GROUP BY area, month;
```

3. 使用 SQLAlchemy 在後端建立與資料庫的連接，並將查詢結果格式化為 JSON 傳回前端。

- **後端功能：**
 - **數據統計：**按違規類型、地區和時間段統計違規數據。
 - **數據篩選：**支持按日期範圍、地點或違規類型篩選數據。
 - **數據輸出：**提供 RESTful API，返回 JSON 格式數據供前端使用。

核心科技與必備知識

開發環境

- **系統要求：**
 - 作業系統：Windows 或 macOS。
 - 記憶體：8GB RAM 以上（建議 16GB 以提升效能）。
 - 硬碟：50GB 可用空間。

開發工具

1. **Python：**3.8 版本，用於撰寫後端邏輯與數據處理。
 - 推薦使用 Anaconda 進行 Python 環境管理。
2. **Git：**版本控制工具，便於團隊協作。
 - 裝設 GitHub Desktop 為可視化界面。
3. **VSCode：**主要的程式編輯器。
 - 推薦擴充工具：
 - Python：支援 Python 語法高亮與調試。
 - Docker：用於管理容器化應用。
 - SQL Formatter：改善 SQL 語法的可讀性。

技術棧

- **Airflow** :
 - 用於排程與監控違規數據的處理工作流（如每日更新儀表板數據）。
 - **PostgreSQL** :
 - 資料庫管理系統，處理儀表板需要的大量結構化數據。
 - 支援地理數據處理，適合分析台北市區域違規數據分布。
 - **Python** :
 - 作為主要後端開發語言，負責數據清理與統計邏輯。
 - 套件包括：
 - **Pandas**：用於數據處理與清理。
 - **GeoPandas**：支持地理空間數據分析。
 - **SQLAlchemy**：用於與 PostgreSQL 資料庫交互。
-

開發步驟

1. 設定環境：
 - 安裝 Python 3.8 與 Anaconda。
 - 安裝 PostgreSQL 資料庫，建立 `EventBasicInfo` 等表結構。
 2. 撰寫後端 API：
 - 使用 Flask 或 FastAPI 建立 RESTful API。
 - 整合 SQLAlchemy 進行資料庫操作。
 3. 設計前端界面：
 - 在 React.js 中使用 Recharts 建立違規分析圖表。
 - 整合 Axios，用於向後端發送請求並渲染數據。
 4. 部署系統：
 - 使用 Docker 容器化部署後端服務。
 - 利用 Airflow 自動化數據更新排程。
-

技術整合示例

後端 API 示例

```

from flask import Flask, jsonify
from sqlalchemy import create_engine, text

app = Flask(__name__)
engine = create_engine('postgresql://user:password@localhost:5432/violation_db')

@app.route('/api/violations', methods=['GET'])
def get_violations():
    query = text("""
        SELECT
            area,
            COUNT(*) AS total_events,
            COUNT(CASE WHEN violation_type = '超速' THEN 1 EN
D) AS speeding,
            COUNT(CASE WHEN violation_type = '闖紅燈' THEN 1 EN
D) AS red_light
        FROM EventBasicInfo
        GROUP BY area
    """)
    with engine.connect() as conn:
        result = conn.execute(query).fetchall()
        return jsonify([dict(row) for row in result])

if __name__ == '__main__':
    app.run(debug=True)

```

總結

此開發計畫結合前端的 Recharts 視覺化與後端的 SQL 聚合查詢，提供高效的數據分析儀表板，並支持交通管理單位進行決策分析。若需針對其他需求進一步補充，請隨時告知！

總結

1. 技術棧：

- 前端：React.js + TailwindCSS + Recharts。
- 後端：Node.js + Fastify/Express。

- 資料庫：MySQL。

2. 整合方式：

- 六大子系統均使用 RESTful API 進行數據交互。
- 前後端分離設計，支持靈活拓展。

3. 優化重點：

- 提高數據查詢效率，避免重複查詢。
- 使用 Gemeni API 加速影像辨識處理。